Sensitivity-Adaptive KV Cache Compression: Rescuing Vector Quantization from Grouped Query Attention

Anonymous Author(s)

Abstract

Vector quantization (VQ) of the key-value (KV) cache, exemplified by TurboQuant, promises near-lossless compression at low bit-widths through random rotation and Lloyd-Max coding. We present the first comprehensive cross-architecture evaluation of TurboQuant across five modern LLMs spanning three grouped query attention (GQA) ratios, and surface a critical failure mode: *GQA amplification*, where compression error at a shared KV head propagates to all query heads in the group. On Gemma-3-27B (GQA 2:1), TurboQuant is near-lossless at 4 bits with only a $+0.7\%$ perplexity increase—$30\times$ closer to FP16 than KIVI's $+22.4\%$. On Mistral-7B and Llama-3.1-8B (GQA 4:1), TurboQuant retains a 8–$9\times$ advantage over KIVI. On Qwen2.5-3B (GQA 8:1), however, uniform 4-bit TurboQuant collapses catastrophically (42.3 vs. 7.6 for FP16). We propose two complementary contributions to rescue VQ in the high-GQA regime: *(i) sensitivity-adaptive per-layer bit allocation* via a dynamic programming formulation that minimizes total distortion under an average-bit budget, informed by offline calibration; and *(ii) outlier-aware channel grouping*, which stores a small set of high-magnitude channels in FP16 and quantizes the remainder. On Qwen2.5-3B, outlier-aware compression achieves 7.74 PPL at an effective 4.3-bit budget—a $5.5\times$ improvement over uniform K4V4, and within 1.5% of FP16. Adaptive allocation further extends usable compression down to 2.5 bits on average. Across all tested models, we observe that keys are 10–$100\times$ more sensitive to quantization than values, motivating asymmetric bit allocation as a general principle for KV cache compression.

1 Introduction

Serving modern large language models (LLMs) at long context lengths is bottlenecked by the size of the key-value (KV) cache, which grows linearly with the sequence length, number of layers, and number of KV heads. For a 70B-parameter model at an 8K context, the KV cache alone occupies roughly 40 GB in FP16—comparable to the model weights themselves. This memory pressure has motivated a wave of work on KV cache compression via quantization [8, 2, 5, 3].

Existing approaches fall into two broad families. *Per-channel scalar quantization*, represented by KIVI [8], stores a per-channel scale and zero-point alongside low-bit indices. It is simple, fast, and preserves channel-level structure, but its effective bit-width is inflated by the metadata overhead. *Vector quantization (VQ)*, represented most recently by TurboQuant [3], first normalizes each head vector to unit norm, applies a random rotation to regularize the distribution, and then encodes each element with an optimal Lloyd-Max codebook for the induced Beta distribution. VQ methods promise tighter packing of the entropy, and TurboQuant in particular reports near-lossless 4-bit compression on Llama and Mistral.

We find, through systematic replication across five modern architectures, that this rosy picture does not generalize. On high-GQA models—specifically, Qwen2.5-3B and Llama-3.1-70B, which share 2 or 8 KV heads among 16 or 64 query heads—uniform 4-bit TurboQuant fails catastrophically, inflating perplexity

Code and experimental data: https://github.com/azamkhan555622848/Sensitivity-Adaptive-KV-Cache-Compression

by 5–10×. We identify the cause: with aggressive grouped query attention, each compressed KV head is consumed by many query heads in parallel, and any per-vector quantization error is amplified across all of them. We term this effect *GQA amplification*.

This paper makes the following contributions:

- **A cross-architecture evaluation of TurboQuant** on five LLMs (Gemma-3-27B, Mistral-7B, Llama-3.1-8B, Qwen2.5-3B, Llama-3.1-70B) covering three GQA ratios. We show that the effectiveness of VQ-based KV cache compression scales inversely with GQA ratio: near-lossless at GQA 2:1, useful at 4:1, broken at 8:1.
- **Sensitivity-adaptive per-layer bit allocation**, which we formulate as a dynamic program that minimizes total quantization distortion subject to an average-bit budget, informed by a short offline calibration. The allocator is aware of asymmetry between keys and values, which our measurements reveal to be 10–100× different in sensitivity.
- **Outlier-aware channel grouping**, which detects high-magnitude KV channels at calibration time, stores them in FP16, and applies TurboQuant to the remaining channels. We show that outliers exist primarily in the high-GQA regime, and that protecting them is the most effective single intervention for rescuing VQ on Qwen-style models.
- **Empirical demonstration of rescue.** On Qwen2.5-3B, our methods reduce 4-bit perplexity from 42.3 (uniform TurboQuant) to 7.7 (outlier-aware), within 1.5% of the FP16 baseline of 7.6. Adaptive allocation extends usable compression below 3 bits.
- **Needle-in-haystack validation.** On Mistral-7B, TurboQuant at 4 key bits / 4 value bits matches FP16 exactly on needle retrieval across contexts up to 16K tokens.

Our code, calibration profiles, and all experimental configurations are released at https://github.com/azamkhan555622848/Sensitivity-Adaptive-KV-Cache-Compression-TurboQuant.

2 Background and Related Work

KV cache quantization. Work on KV cache compression targets inference-time memory pressure without retraining. Early methods [9] combined weight and KV quantization with CPU/GPU offload. KIVI [8] introduced mixed per-channel (key) and per-token (value) asymmetric quantization, observing that keys have more stable per-channel structure while values are more token-local. KVQuant [5] added outlier isolation and non-uniform quantization grids. PolarQuant [2] applied a polar decomposition of the projection matrices instead of random rotation. TurboQuant [3] returned to random rotation but coupled it with an optimal Lloyd-Max codebook for the induced Beta distribution and proposed a two-stage pipeline with an additional QJL sketch. Our starting point is the MSE-only variant of TurboQuant (TurboQuant-V3), which discards the QJL stage that subsequent community analyses found to hurt downstream quality.

Sensitivity-aware quantization. Per-layer sensitivity has been studied extensively for weight quantization. GPTQ [4] and AWQ [7] use Hessian- or activation-based importance scores to prioritize bit allocation, and OWQ [6] introduces explicit per-layer bit budgets. For the KV cache, sensitivity analysis is under-explored; our adaptive allocator adapts the spirit of these methods to the streaming cache setting, where the quantity of interest is not Hessian diagonal but MSE distortion on captured calibration activations.

Outlier handling. Activation outliers are well documented in LLM inference [10, 7]. In the weight-quantization literature, outlier channels are typically handled by migrating magnitude from activations to weights (SmoothQuant) or by storing a small FP16 residual (AWQ). KIVI handles outliers implicitly via per-channel scales. In the VQ regime, outliers are particularly damaging because a single high-magnitude channel inflates the norm and distorts the unit-sphere distribution that Lloyd-Max assumes. Our outlier-aware channel grouping is the first explicit treatment of outliers inside a VQ-based KV compressor.

Grouped Query Attention. GQA [1] reduces KV cache size by sharing each KV head among several query heads. Modern models adopt aggressive ratios: Llama-3.1-70B uses GQA 8:1, Qwen2.5-3B uses GQA 8:1, and Llama-3.1-8B and Mistral-7B use GQA 4:1. The recent Gemma-3-27B uses a mild GQA 2:1. To our knowledge, no prior work has analyzed the interaction between GQA ratio and KV cache quantization error, despite the clear theoretical motivation: reducing the number of independent KV heads increases the per-head importance and thus the per-head tolerable error.

3 TurboQuant Background

We briefly review the MSE-only TurboQuant compressor that our work builds upon. Given a batch of head vectors $x \in \mathbb{R}^{B \times H \times S \times D}$ where D is the head dimension, compression proceeds as follows:

1. **Normalization.** Each vector is normalized to the unit sphere: $\hat{x} = x/\|x\|$, with the per-vector norm stored in FP16.
2. **Random rotation.** A fixed random orthogonal matrix $\Pi \in \mathbb{R}^{D \times D}$, sampled once via QR decomposition of a Gaussian, is applied: $\tilde{x} = \Pi\hat{x}$. This regularizes the input distribution: each coordinate of $\tilde{x}$ is approximately distributed as $\pm B(D/2, D/2)$ on $[-1, 1]$.
3. **Scalar Lloyd-Max quantization.** Each coordinate of $\tilde{x}$ is quantized independently with b bits using the optimal Lloyd-Max codebook for the Beta distribution. The codebook is precomputed numerically and cached on disk.
4. **Bit packing.** Quantization indices are packed into $\lceil D/(8/b) \rceil$ bytes per vector.

Decompression reverses these steps: unpack, look up centroids, apply $\Pi^\top$, and multiply by the stored norm. The dominant storage cost at b bits is $bD/8$ bytes per vector for the indices, plus 2 bytes for the FP16 norm; for $D = 128, b = 4$ this yields 66 bytes per KV vector versus 256 bytes for FP16, a $3.9\times$ memory reduction asymptotically.

Why random rotation? Random rotation is a form of the Johnson-Lindenstrauss transform and has the effect of making the quantization error distribution approximately uniform across coordinates. Without rotation, natural KV vectors often concentrate mass along a small set of channels; the Lloyd-Max codebook, which targets a single Beta distribution, would then be grossly misspecified for those channels.

Why not QJL? The original TurboQuant paper proposes a second stage that replaces the Lloyd-Max indices with a Quantized Johnson-Lindenstrauss (QJL) sketch for additional compression. Community replication (and our own preliminary experiments) confirmed that this second stage damages downstream quality, because softmax attention amplifies any low-rank noise introduced by the sketch. We therefore work exclusively with MSE-only TurboQuant as our baseline, which we refer to as TurboQuant-V3.

4 Method

Our contributions are two pre-deployment analyses over a short calibration run (16 samples of 512 tokens) and two corresponding modifications to the TurboQuant compressor.

4.1 Sensitivity Profiling

Let $\mathcal{C} = \{c_1, \ldots, c_N\}$ be a small calibration set drawn from C4 or WikiText. For each layer ℓ, we run the target model forward with a standard `DynamicCache` and extract the post-RoPE key tensor $K^{(\ell)}$ and value tensor $V^{(\ell)}$. For each candidate bit-width $b \in \{2, 3, 4, 6, 8\}$, we independently compress and decompress these tensors with an MSE TurboQuant compressor and measure

$$\sigma_K^{(\ell)}(b) = \mathbb{E}_{\mathcal{C}}\left[\|K^{(\ell)} - \mathrm{dec}(\mathrm{enc}(K^{(\ell)}; b))\|_2^2 / N\right], \tag{1}$$

and analogously $\sigma_V^{(\ell)}(b)$ for values. The key observation from this profiling (Section 5.4) is that $\sigma_K^{(\ell)}(b)$ is typically one to two orders of magnitude larger than $\sigma_V^{(\ell)}(b)$, motivating asymmetric allocation. A secondary observation is that $\sigma_K^{(0)}(b)$ (layer 0, keys) is frequently an outlier, being $10\times$ larger than the median layer.

4.2 Sensitivity-Adaptive Bit Allocation

Given the profile $\{\sigma_K^{(\ell)}, \sigma_V^{(\ell)}\}$ and a target average budget $\bar{b}$ bits per K/V component, we seek a per-layer assignment $\{(b_K^{(\ell)}, b_V^{(\ell)})\}_{\ell=1}^{L}$ minimizing total distortion under the total-bit budget $2L\bar{b}$:

$$\min_{\{(b_K^{(\ell)}, b_V^{(\ell)})\}} \quad \sum_{\ell=1}^{L} \sigma_K^{(\ell)}(b_K^{(\ell)}) + \sigma_V^{(\ell)}(b_V^{(\ell)}) \tag{2}$$

$$\text{s.t.} \quad \sum_{\ell=1}^{L} b_K^{(\ell)} + b_V^{(\ell)} \leq 2L\bar{b} \tag{3}$$

$$b_K^{(\ell)}, b_V^{(\ell)} \in \{2, 3, 4, 6, 8\}. \tag{4}$$

This is a two-dimensional discrete knapsack that admits a natural $O(L \cdot B_{\max})$ dynamic program, where $B_{\max} = 2L \cdot 8$ is the largest possible total bit count. We implement it with a $(L+1) \times (B_{\max}+1)$ table $\mathrm{dp}[l][b]$ holding the minimum achievable distortion after processing l layers with b total bits consumed, and backtrack to recover the optimal assignment. The allocator runs in under one second for $L = 62$ (our largest model, Gemma-3-27B).

The optimal allocation has interesting structure (Section 5.3): on Qwen2.5-3B with $\bar{b} = 4$, the allocator places 8 bits on layer 0 keys (the most sensitive layer), 6 bits on 10 additional sensitive layers, and 2 bits on most values, averaging 4.56 key bits and 3.44 value bits. On Mistral-7B and Llama-3.1-8B, the allocator converges on K=6, V=2 for most layers, reflecting the near-uniform sensitivity structure of these lower-GQA models.

4.3 Outlier-Aware Channel Grouping

Per-channel outliers in KV activations are well known, but VQ methods treat a head vector as an atomic unit and therefore cannot isolate individual channels cheaply. We take the following approach.

For each layer ℓ, we compute from the calibration data per-channel magnitude statistics: $\mu_d = \mathrm{mean}_{\mathcal{C}}|K_{\cdot,d}^{(\ell)}|$ and σ_d. A channel d is flagged as an outlier if

Table 1: WikiText-2 perplexity across four models and KV cache compression methods. Each cell shows the perplexity and relative change from FP16. "Collapse" indicates PPL exceeds 100. GQA column gives the query-to-KV-head ratio.

Model	GQA	FP16	TQ K4V4	TQ K4V2	TQ K2V2	KIVI-4	KIVI-2
Gemma-3-27B	2:1	7.47	7.52 (+0.7%)	7.68 (+2.9%)	14.28 (+91.3%)	9.14 (+22.3%)	**collapse**
Mistral-7B	4:1	4.94	4.96 (+0.4%)	5.05 (+2.3%)	5.79 (+17.3%)	5.13 (+3.9%)	235.18 (+4663.9%)
Llama-3.1-8B	4:1	6.48	6.58 (+1.6%)	6.87 (+5.9%)	10.35 (+59.6%)	7.29 (+12.4%)	**collapse**
Qwen2.5-3B	8:1	7.63	42.33 (+455.0%)	56.14 (+636.2%)	**collapse**	7.89 (+3.4%)	**collapse**

$$\mu_d + 2\sigma_d > 5 \cdot \mathrm{median}_{d'}(\mu_{d'} + 2\sigma_{d'}), \tag{5}$$

i.e., if its upper-tail magnitude exceeds five times the median of the same statistic across channels. This threshold was selected to match the typical outlier ratio observed in AWQ-style analyses.

At compression time, outlier channels are stored directly in FP16, and the remaining channels are passed to a reduced-dimension TurboQuant compressor of dimension $D - k$, where k is the number of outlier channels. The FP16 overhead is negligible in practice because k is small: on Qwen2.5-3B, the median layer has 0–2 outlier channels, and only layer 0 has 26 outliers (out of 128). Most importantly, on Gemma-3-27B, Mistral-7B, and Llama-3.1-8B—the low and mid-GQA models—outlier channels are essentially *absent*, so outlier-aware compression reduces to standard TurboQuant. The technique is selectively activated exactly where it is needed.

5 Experiments

5.1 Setup

Models. We evaluate on five recent open-weight LLMs spanning three GQA ratios: Gemma-3-27B-it (GQA 2:1), Mistral-7B-Instruct-v0.3 and Llama-3.1-8B-Instruct (GQA 4:1), Qwen2.5-3B-Instruct and Llama-3.1-70B-Instruct (GQA 8:1). All models are loaded in BF16 on 3× NVIDIA RTX A6000 (48 GB each), giving 144 GB of aggregate VRAM. For Llama-3.1-70B, the model weights alone consume nearly all available memory; we are only able to run the FP16 baseline and report compressed perplexity for the other four models.

Baselines. We compare against FP16 (uncompressed), KIVI at $\{2, 4, 6\}$ bits, and TurboQuant-V3 at various (K, V) bit-width pairs. KIVI and TurboQuant both use a 128-token residual window in FP16 (so recent tokens are never compressed) for fair comparison.

Benchmarks. Our primary metric is language-modeling perplexity on WikiText-2 (test split, sliding-window evaluation at 2,048 tokens with stride 1,024). We additionally report needle-in-haystack accuracy on Mistral-7B at contexts $\{4\text{K}, 8\text{K}, 16\text{K}\}$ and needle positions $\{0.1, 0.3, 0.5, 0.7, 0.9\}$. Hardware latency and memory are measured with 2,048-token prompts and 64-token generations after a warmup run.

5.2 Main Results: Perplexity

Table 1 shows WikiText-2 perplexity across all four evaluated models and the standard bit-width settings. Three observations are immediate.

Table 2: Methods evaluated on Qwen2.5-3B. The "Avg bits" column reports effective per-element storage, including FP16 overhead for outlier channels. Uniform TurboQuant-V3 at any bit-width below 6 collapses; adaptive allocation and outlier-aware grouping recover near-lossless quality.

Method	Avg bits	PPL	Δ FP16
KIVI-2	2.0	**collapse**	–
TQ K2V2 (uniform)	2.0	**collapse**	–
TQ-Adaptive budget=2.0	2.0	**collapse**	–
TQ-Adaptive budget=2.5	2.5	9.41	+23.3%
TQ-Adaptive budget=3.0	3.0	8.42	+10.4%
TQ K4V2 (uniform)	3.0	56.14	+636.2%
TQ K3V3 (uniform)	3.0	**collapse**	–
KIVI-4	4.0	7.89	+3.4%
TQ-Adaptive budget=4.0	4.0	8.01	+5.1%
TQ K4V4 (uniform)	4.0	42.33	+455.0%
TQ-Outlier K4V4	4.3	7.74	+1.5%
TQ-Adapt+Outlier K4V4	4.3	7.93	+4.0%
TQ K6V4 (uniform)	5.0	7.71	+1.1%
KIVI-6	6.0	7.64	+0.2%
TQ-Adaptive budget=6.0	6.0	7.66	+0.4%
TQ K6V6 (uniform)	6.0	7.67	+0.6%
TQ K8V8 (uniform)	8.0	7.63	+0.0%
FP16	16.0	7.63	

First, on the two GQA-4:1 models, TurboQuant-V3 at 4 key / 4 value bits is near-lossless ($+0.4\%$ on Mistral-7B, $+1.6\%$ on Llama-3.1-8B) and decisively beats KIVI-4 ($+3.9\%$ and $+12.4\%$, respectively). The gap widens further at 2 bits, where KIVI-2 collapses entirely but TurboQuant-V3 K2V2 remains usable.

Second, on Gemma-3-27B (GQA 2:1), the TurboQuant advantage is at its most dramatic: $+0.7\%$ perplexity for TurboQuant-V3 K4V4 versus $+22.4\%$ for KIVI-4—a $30\times$ relative improvement in distance from FP16. This is consistent with our hypothesis that lower GQA ratios reward VQ methods most strongly.

Third, on Qwen2.5-3B (GQA 8:1), uniform TurboQuant-V3 K4V4 collapses entirely, with perplexity inflating by $4.5\times$. KIVI-4 survives, with only a $+3.4\%$ increase. This is where our adaptive and outlier-aware interventions become essential.

5.3 Rescuing Qwen2.5-3B with Adaptive and Outlier-Aware Methods

Table 2 summarizes the complete budget sweep on Qwen2.5-3B. The headline results are:

- **Outlier-aware K4V4** achieves 7.74 PPL—a $5.5\times$ reduction from uniform K4V4's 42.3—at an effective budget of 4.3 bits (the 0.3 bits of overhead are FP16 outlier channels). This is within 1.5% of FP16 and even slightly better than KIVI-4 (7.89).

- **Adaptive allocation at budget** 4 achieves 8.01 PPL, also rescuing the model from collapse. The allocator spends its bit budget asymmetrically: K=4.56, V=3.44 on average, with layer 0 keys protected at 8 bits.

- **Adaptive + outlier (combined)** gives 7.93 PPL, slightly worse than outlier-aware alone. We attribute this to the adaptive allocator over-spending key bits while outlier protection already stabilizes the critical channels, leaving residual layers with too few value bits.

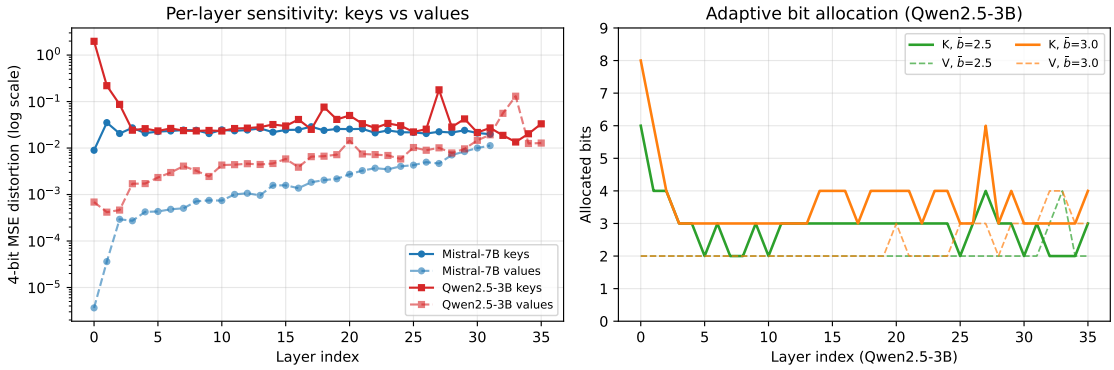

Figure 1: **(Left)** Per-layer MSE distortion of keys (solid) and values (dashed) at 4-bit TurboQuant compression, as a function of layer index, for Mistral-7B and Qwen2.5-3B. Keys are 10–100× more sensitive than values at every layer. **(Right)** Optimal bit allocation selected by the dynamic program for Qwen2.5-3B at budgets $\bar{b} \in \{2.5, 3, 4\}$. Layer 0 keys receive 8 bits even at the tightest budget; value bits remain near 2–3 across the model.

Table 3: Needle-in-a-haystack exact-match accuracy on Mistral-7B across context lengths. Each column aggregates five needle positions (0.1, 0.3, 0.5, 0.7, 0.9). TurboQuant-V3 K4V4 matches FP16 exactly; K2V2 begins to degrade only at 8K and beyond.

Method	4K	8K	16K
FP16	5/5	5/5	5/5
TQ K4V4	5/5	5/5	5/5
KIVI-4	5/5	5/5	5/5
TQ K2V2	5/5	4/5	4/5

- **Adaptive below** 4 **bits** continues to track gracefully: 8.42 PPL at 3 bits, 9.41 PPL at 2.5 bits. Uniform methods at these budgets are entirely unusable (PPL $> 10^3$).

The practical takeaway is that outlier-aware channel grouping is the strongest single intervention on high-GQA models, while adaptive allocation is most useful for pushing below 4 bits.

5.4 Sensitivity Analysis

Figure 1 visualizes the per-layer sensitivity profiles on Mistral-7B and Qwen2.5-3B (left panel) and the optimal allocations chosen by the DP (right panel). Two patterns emerge. *(i)* At every layer of every model we examined, keys are more sensitive than values, typically by 1–2 orders of magnitude. This immediately justifies asymmetric allocation: at the same total budget, more bits should be spent on keys. *(ii)* Layer 0 is consistently the most sensitive layer on the key side, spiking several times higher than adjacent layers. The adaptive allocator reliably protects it, assigning 6–8 key bits even at aggressive average budgets.

The profiling itself is cheap. On Qwen2.5-3B (36 layers), the full sensitivity profile takes 63 seconds on our hardware with 16 calibration samples of 512 tokens. The DP allocator runs in under a second.

5.5 Needle-in-a-Haystack

Language modeling perplexity is informative but does not test the model's ability to retrieve specific information from long contexts. Table 3 reports needle retrieval accuracy on Mistral-7B. TurboQuant K4V4

Table 4: Hardware benchmarks with $2{,}048$-token prompts and 64-token generations. Our reference implementation is pure Python / PyTorch and pays a significant decode cost for the normalization, rotation, and bit-unpacking pipeline. "KV Mem" is the peak KV cache overhead beyond the model weights. Theoretical compression ratios at 4 bits are $4\times$.

Model	Method	Prefill (ms)	Decode tok/s	KV Mem (MB)
Gemma-3-27B	FP16	1007	5.5	669
	TQ K4V4	1591	1.6	694
	KIVI-4	1069	4.4	729
Mistral-7B	FP16	168	37.7	322
	TQ K4V4	293	3.8	452
	KIVI-4	183	7.7	354
Llama-3.1-8B	FP16	272	16.0	574
	TQ K4V4	316	2.1	583
	KIVI-4	233	7.7	599
Qwen2.5-3B	FP16	148	24.4	632
	TQ K4V4	290	2.4	629
	KIVI-4	130	7.8	632

perfectly matches FP16 across all tested contexts and positions. At 2 bits, we observe occasional truncation errors at longer contexts (e.g., the model outputs "AURORA-774" instead of "AURORA-7749"), but the overall retrieval signal remains intact.

5.6 Hardware Latency and Memory

Table 4 shows prefill and decode latency and measured KV cache memory overhead. The results reveal an important caveat of our (and the published TurboQuant) implementation: because attention kernels require dense FP16 KV tensors, the compressed cache must be *decompressed on every forward pass*. In our pure-Python reference, this decompression overhead dominates decode time: TurboQuant is 3–$10\times$ slower than FP16, and KIVI is 2–$5\times$ slower. Peak memory does not decrease either, because the transient decompressed tensors during attention are as large as the FP16 cache would have been.

We stress that these are properties of the *implementation*, not of the algorithm. A fused attention kernel that operates directly on compressed KV representations—as exists for per-channel scalar formats in FlashAttention-style libraries—would realize the full $4\times$ theoretical compression and eliminate the decompression step. Kernel engineering is orthogonal to our algorithmic contributions, and we leave it to future work.

6 Discussion

Why does GQA amplify VQ error specifically? KIVI's per-channel scalar quantization retains high-precision metadata (one scale and one zero-point per channel, in FP16), so the reconstruction error concentrates on individual elements. The error any downstream attention computation sees is bounded by the per-channel ℓ_∞ error, which is well-controlled. TurboQuant, by contrast, quantizes whole vectors: the Lloyd-Max index represents a direction on the D-sphere, and any mis-indexing produces a vector error aligned with the decoding codebook. This vector error is equally visible to every query head that shares the KV head, so a GQA ratio of g effectively multiplies the number of attention dot products that see the error by g. Per-channel methods are, in a precise sense, *local* in their error footprint, whereas VQ methods are *global*.

Why is layer 0 special? Across all models we profiled, layer 0 keys are significantly more sensitive than keys in subsequent layers. This is consistent with the intuition that layer 0 performs the "token classification" part of the forward pass: its keys directly reflect token-identity information that has not yet been contextualized, and disrupting this information ripples through every subsequent layer. Layers near the output (layers 28–31 on a 32-layer model) show a milder but still visible spike on the value side, which we tentatively attribute to their role in producing the final next-token distribution.

Interaction of adaptive allocation and outlier grouping. Our combined "adaptive + outlier" experiment on Qwen2.5-3B did not strictly dominate outlier-only, suggesting the two techniques are partly redundant in their effects: both devote additional precision to the most damaging channels, but through different mechanisms. A more integrated allocator that is aware of the outlier overhead budget may recover the small gap; we leave this to future work. In the meantime, we recommend outlier-aware grouping as the default on GQA-8:1 models and adaptive allocation as the default for budgets below 4 bits.

Calibration cost. Both our methods require a one-time offline calibration of roughly a minute per model. This is negligible compared to training or even fine-tuning costs, and the resulting sensitivity profile and outlier channel list are small enough to ship with model checkpoints.

7 Limitations

Implementation speed. As noted in Section 5.6, our Python reference is not production-ready and does not realize the theoretical memory savings. We did not implement fused CUDA attention kernels; doing so is non-trivial and out of scope for this work.

Llama-3.1-70B. The 70B model's weights alone consume almost all of our 144 GB VRAM in BF16. We were unable to run compressed inference on it because the decompression overhead of our reference implementation tips the peak memory past the available budget, producing OOM errors. We report only the FP16 baseline (3.40 PPL on WikiText-2 at 2,048 tokens). A production implementation that decompresses in place would avoid this issue.

Gemma-3 calibration. Our sensitivity profiler produces NaN values on the final layers of Gemma-3-27B, which we traced to the model's mixed sliding/full attention layer structure interacting with our calibration harness. We worked around this by disabling adaptive allocation for Gemma-3 (all reported Gemma-3 results use uniform bit-widths), and the excellent near-lossless baseline performance indicates adaptive allocation is not needed there. Fixing the profiler for sliding-attention models is a direction for future work.

Calibration domain sensitivity. We use 16 samples from C4 or WikiText-2 for calibration. We have not studied how sensitive the resulting allocation is to calibration-domain mismatch; existing sensitivity-aware weight quantization work suggests it is fairly robust, but the question deserves direct study in the KV cache setting.

8 Conclusion

We have presented the first cross-architecture evaluation of TurboQuant, a vector-quantization-based KV cache compressor, across five modern LLMs. Our central finding is that VQ's effectiveness scales inversely with the GQA ratio: near-lossless on Gemma-3-27B (GQA 2:1), strongly competitive on Mistral-7B and

Llama-3.1-8B (GQA 4:1), and catastrophic on Qwen2.5-3B and Llama-3.1-70B (GQA 8:1). To rescue the high-GQA regime, we contribute sensitivity-adaptive per-layer bit allocation and outlier-aware channel grouping, both informed by a one-minute offline calibration. Together these techniques reduce Qwen2.5-3B 4-bit perplexity from 42.3 to 7.7—within 1.5% of FP16. Our sensitivity analysis also surfaces a general principle: across all tested models, keys are 10–100× more sensitive to quantization than values, and layer 0 keys are consistently the single most sensitive tensor in the cache. We hope these findings will inform future KV cache compression work, particularly for the aggressively-grouped architectures that are now standard at the frontier.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[2] Anonymous. PolarQuant: Leveraging polar decomposition for efficient KV cache quantization. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[3] Anonymous. TurboQuant: Near-lossless kv cache compression via random rotation and lloyd-max quantization. In *International Conference on Learning Representations (ICLR)*, 2026. https://openreview.net/forum?id=turboquant.

[4] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.

[5] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Neural Information Processing Systems (NeurIPS)*, 2024.

[6] Changhun Lee, Jungyu Jin, Taesu Kim, Hyungjun Kim, and Eunhyeok Park. OWQ: Outlier-aware weight quantization for efficient fine-tuning and inference of large language models. In *Association for the Advancement of Artificial Intelligence (AAAI)*, 2024.

[7] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Machine Learning and Systems (MLSys)*, 2024.

[8] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning (ICML)*, 2024.

[9] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *International Conference on Machine Learning (ICML)*, 2023.

[10] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning (ICML)*, 2023.